МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по практической работе №1
по дисциплине «Интеллектуальные системы анализа геопространственных данных»
Тема: Введение в геопространственный анализ: координаты, проекции и визуализация

Студент гр. 1304 _____ Гирман А.В.

Преподаватель _____ Уткина О.Н.

Санкт-Петербург
2026

Цель работы.

Освоение фундаментальных навыков работы с геопространственными данными и системами координат, необходимых для решения задач экологического мониторинга.

Задачи:

1. Освоить базовые понятия геодезических систем координат и проекций.
2. Научиться работать с векторными геоданными в формате GeoJSON и Shapefile.
3. Освоить преобразование координат между различными системами (СК42, WGS84, UTM).
4. Научиться визуализировать геоданные на интерактивных картах.
5. Понять важность правильного выбора проекции для экологических расчетов.
6. Заложить основу для сквозного проекта по мониторингу качества воды.

Задание варианта 3.

Регион: Астраханская область

Водный объект: Дельта реки Волги

Координаты (примерные границы): 45.0–46.0° с.ш., 47.0–49.0° в.д.

Теоретический минимум.

Ключевые понятия:

- **Геодезическая система координат** — модель Земли, задающая способ описания положения точек в пространстве. Включает параметры эллипсоида (форму и размеры Земли), начало отсчета и ориентацию осей. Пример: WGS84 — всемирная система, используемая GPS.

- **Проекция** — математический способ переноса поверхности эллипсоида на плоскость. Приводит к искажениям длин, площадей, углов или форм. Выбор проекции зависит от цели карты (например, равноугольная для навигации, равновеликая для расчета площадей).
- **EPSG-код** — уникальный идентификатор системы координат в международной базе данных. Позволяет однозначно указать проекцию и датум. Примеры: EPSG:4326 = WGS84 (географические координаты), EPSG:28407 = СК-42 для зоны 7 (проекция Гаусса-Крюгера).
- **Датум** — набор параметров, определяющий положение и ориентацию эллипсоида относительно Земли. От него зависит точность координат. Например, СК-42 и WGS84 используют разные датумы, что приводит к сдвигам до сотен метров.
- **UTM (Universal Transverse Mercator)** — универсальная поперечная проекция Меркатора. Разделяет Землю на 60 зон шириной 6° долготы. Часто используется для спутниковых снимков и инженерных расчетов, так как минимизирует искажения в пределах одной зоны.

Системы координат

Географическая система координат — описывает положение точки на поверхности эллипсоида с помощью угловых величин: широты и долготы.

Система	Описание	Применение
WGS84	Всемирная геодезическая система 1984 года. Стандарт для GPS и большинства современных геоданных. Использует глобальный эллипсоид WGS84, центр которого совпадает с центром масс	GPS, Google Maps, большинство современных геоданных

	Земли	
СК-42	Система координат 1942 года, созданная в СССР. Основана на эллипсоиде Красовского	Старые топографические карты, архивы

Проекция — математический способ переноса поверхности эллипсоида на плоскость. Любая проекция вносит искажения:

- равноугольные — сохраняют углы, искажают площади;
- равновеликие — сохраняют площади, искажают углы;
- равнопромежуточные — сохраняют расстояния по определенным направлениям.

UTM (Universal Transverse Mercator) — зональная проекция, делит Землю на 60 зон по 6° долготы. В пределах одной зоны искажения минимальны.

Основы преобразования координат

Преобразование между СК-42 и WGS84 требует учета параметров датума, так как эллипсоид Красовского (СК-42) отличается от эллипсоида WGS84.

Основные шаги:

1. Определение исходных и целевых EPSG-кодов.

   Например:

   - исходные: EPSG:28407 (СК-42, зона 7);

   - целевые: EPSG:4326 (WGS84).

2. Применение семипараметрического преобразования (формула преобразования Бурса— Вольфа «Bursa—Wolf transformation»):

$$\begin{cases} \Delta X = \Delta X_0 + \omega_z \cdot Y - \omega_y \cdot Z + m \cdot Z \\ \Delta Y = \Delta Y_0 - \omega_z \cdot X + \omega_x \cdot Z + m \cdot Y \\ \Delta Z = \Delta Z_0 + \omega_y \cdot X - \omega_x \cdot Y + m \cdot Z \end{cases}$$

где ΔX0, ΔY0, ΔZ0 — сдвиг начала координат, ωx, ωy, ωz — углы поворота, m — масштабный коэффициент.

3. Перепроецирование, если требуется плоская система (например, UTM).

Выполнение работы.

1. Знакомство с GeoPandas

1.1 В загруженном слое *ne_110m_admin_0_countries* одна строка соответствует одной стране, и у каждой строки только одна геометрия, но территории стран бывают устроены по-разному, поэтому типы геометрий разные: POLYGON (148 стран) и MULTIPOLYGON (29 стран).

В POLYGON попадают страны с территорией масштаб которых представляет собой один сплошной участок суши с одним внешним контуром.

В MULTIPOLYGON попадают страны с территорией, которая состоит из нескольких несмежных частей: материковой части и островов, эксклавов или отдельных территорий.

1.2 Изучим структуру данных.

Столбцы:

- NAME – хранит название страны;
- POP_EST – хранит оценку численности населения;
- GDP_MD – хранит ВВП;
- geometry – хранит форма границы каждой страны в виде объектов (долгота, широта);

Для описания территории России используется MULTIPOLYGON с 14 полигонами. Загруженную карту мира можно увидеть рисунке (см. рис 1).

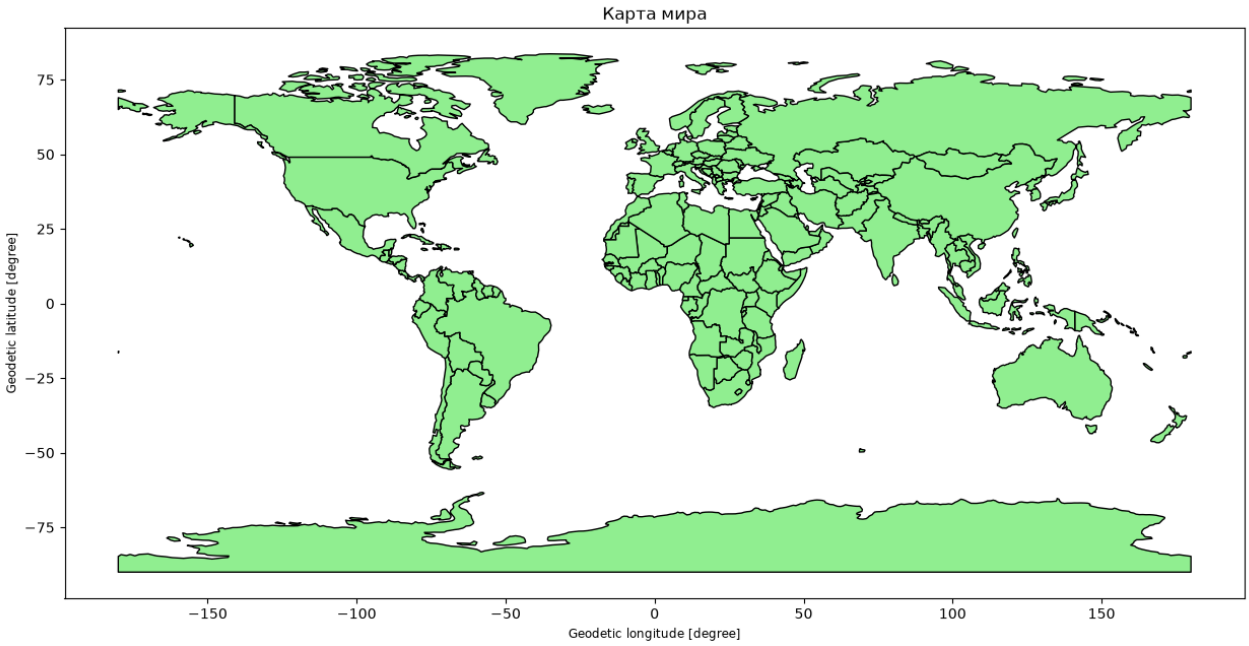


Рисунок 1 – Карта мира

1.3 Создан класс *WaterQualityPipeline*. Он объединяет этапы обработки данных мониторинга качества воды: загрузку исходных данных, создание точек отбора проб и вывод сводки о текущем состоянии.

Метод *create_sample_points(n_points, region)* изменён под район исследования (согласно варианту 3). Тестовые точки отбора проб воды теперь создаются для Астраханской области. Координаты точек генерируются случайно с равномерным распределением в пределах примерных границ дельты реки Волги: широта: 45.0 - 46.0° с.ш., долгота: 47.0 - 49.0° в.д.

Исходный код представлен в приложении А.

2. Преобразование координат.

2.1 В классе созданы точки в системе СК-42 и с помощью функции эти точки преобразуются в WGS84:

- *create_sk42_points* создаёт тестовые точки в системе координат СК-42 в EPSG:28408 (для Астраханской области зона 8). Метод генерирует случайные значения X (4 970 000 - 5 100 000 м.) и Y (8 660 000 - 9 350 000 м.) и формирует из них *GeoDataFrame*. Объекту

присваивается система координат EPSG:28408. Метод возвращает полученный набор точек.

- *convert_to_wgs84* преобразует точки проб в систему WGS 84 (EPSG:4326) с помощью метода *to_crs*. При перепроецировании пересчитываются только координаты геометрии из метров СК-42 в градусы долготы и широты.

2.2 Для пяти точек отбора проб, созданных в системе СК-42 и преобразованных в WGS 84, рассчитаны расстояния между соседними точками двумя способами:

Первый способ заключается в использовании проекции UTM (зона 39N, EPSG:32639, для Астраханской области). Зона определена автоматически по средним координатам точек с помощью библиотеки *utm*. Координаты в этой проекции заданы в метрах, поэтому расстояние вычисляется напрямую.

Второй способ в географических координатах WGS 84 (EPSG:4326). Расстояние вычислено в градусах и переведено в километры по грубому соотношению, что 1 градус примерно равен 111 км.

В таблице 1 можно увидеть разницу полученных результатов по двум способам.

Таблица 1 – Сравнение методов.

Пара	В UTM (м)	В градусах (км)	Разница (км)
1-2	469510	659.18	189.67
2-3	306490	431.20	124.71
3-4	261776	358.23	96.46
4-5	232537	315.50	82.96

По таблице 1 можно сделать вывод, что максимальная разница между двумя способами достигает 189.67 км, а минимальная 82.96 км. Эта большая разница получается в результате того, что при втором способе 1 градус в любом

направлении считается равным 111 км и не учитывается, что длина градуса долготы уменьшается к полюсам. Поэтому такой расчёт завышает расстояния. Проекция UTM задаёт координаты в метрах и почти не искажает длины. Следовательно, для расчёта расстояний данные нужно перепроецировать в метрическую проекцию с помощью метода *to_crs*.

3. Исследование влияния проекций на площади.

3.1 Расчёт в географической системе WGS 84 (см. рис. 2) даёт площадь вдвое больше реальной: ошибка 112,3 %. Равновеликие проекции Мольвейде и Альберса дают результат, близкий к реальному: отклонение около 1 %.

При запуске метода появилось предупреждение потому, что геометрия России хранится в географической системе координат EPSG:4326, где координаты представляются как углы (градусы долготы и широты), а не метры. Метод *area* считает площадь в квадратных градусах (111 км × 111 км), а длина градуса долготы уменьшается к полюсам. Поскольку Россия расположена в высоких широтах, то перевод, что 1 градус примерно равен 111 км завышает площадь примерно вдвое.

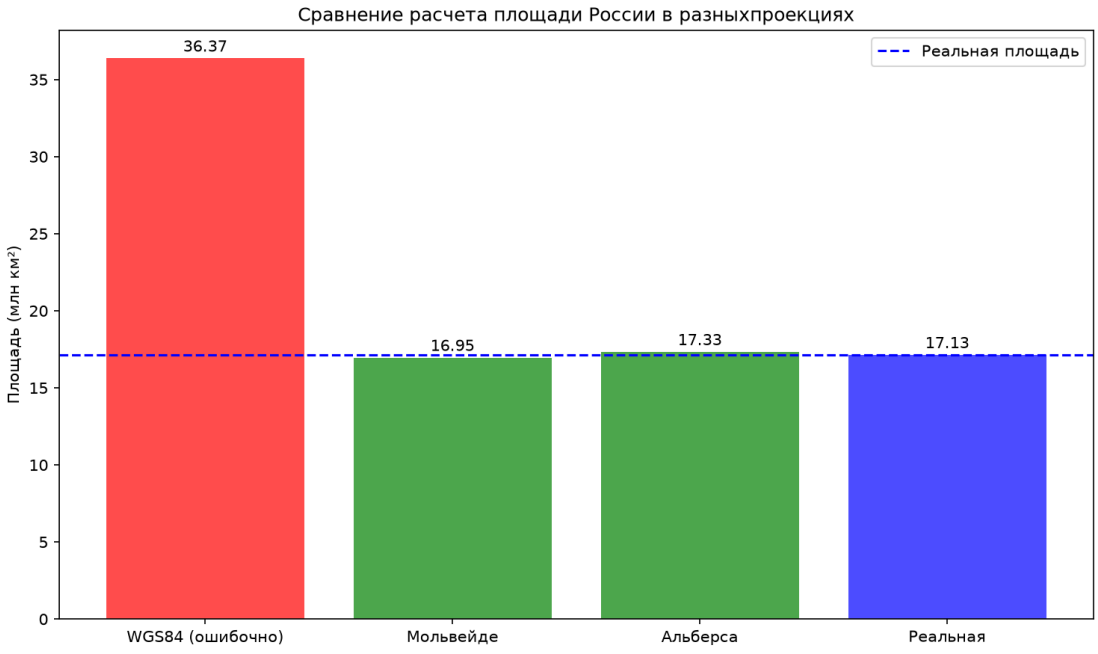


Рисунок 2 – Сравнение площади России в разных проекциях

4. Визуализация в Folium

4.1 Для визуализации точек отбора проб в класс *WaterQualityPipeline* добавлен метод *visualize_samples*. Он вычисляет центр карты, как среднее значение широты и долготы всех точек и создаёт карту с центром в этой точке. Для каждой точки добавляет маркер со всплывающим окном, где показаны идентификатор и название пробы. Сохраняет карту в формате HTML в папку результатов.

На рисунке 3 представлен скриншот интерактивной карты с точками проб.



Рисунок 3 – Интерактивная карта (Астраханская область, радиус 4 км)

4.2 Для анализа территории вокруг точек отбора проб в класс *WaterQualityPipeline* добавлен метод *create_buffer_zones*. Он определяет зону UTM по средним координатам точек и переводит в неё точки, чтобы работать в метрах. Вокруг каждой точки строит буферную зону заданного радиуса, затем переводит буферы обратно в WGS 84 для отображения. На карту добавляются

буферные зоны с подсказкой, где написан идентификатор и название точки, и маркеры точек проб. Карта сохраняется в формате HTML в папку результатов.



Рисунок 4 – Буферные зоны вокруг точек проб (синяя область)

5. Итоговая проверка и сохранение результатов.

5.1 Для сохранения результатов добавлен метод *save_results*. Он собирает основные сведения о пайплайне: название, количество точек проб, систему координат и средние широту и долготу точек. Эти сведения записываются в файл JSON, а сами точки проб в формате GeoJSON.

Выводы.

Освоены базовые навыки работы с геопространственными данными и системами координат, необходимых для решения задач экологического мониторинга. В ходе работы изучены геодезические системы WGS84 и СК-42, проекции UTM, Гаусса-Крюгера, Мольвейде и Альберса. Реализован базовый класс сквозного проекта *WaterQualityPipeline* с методами создания точек отбора проб для дельты Волги, преобразования координат из СК-42 (EPSG:28408) в

WGS84, расчёта расстояний и площадей, построения буферных зон, визуализации в *Folium* и сохранения результатов в форматах *JSON* и *GeoJSON*. Эксперименты показали, что выбор системы координат принципиально влияет на результат. Площадь России, рассчитанная в градусах WGS84, завышена на 112,3 % относительно эталонного значения, а равновеликие проекции Мольвейде и Альберса дают отклонение около 1 %. Приближение, что 1 градус приблизительно равен 111 км завышает расстояния между точками, так как не учитывает сокращение длины градуса долготы с ростом широты. Поэтому для метрических расчётов и построения буферных зон данные необходимо было перепроецировать в метрическую проекцию с помощью UTM (зона 39N для Астраханской области).

ПРИЛОЖЕНИЕ А

Исходный код программы

Файл main.py:

```
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
from water_quality_pipeline import WaterQualityPipeline
import os
import webbrowser


def task_1_1():
    world = gpd.read_file('data/ne_110m_admin_0_countries.shp')
    print("Данные загружены!")
    print(f"Количество стран: {len(world)}")
    print(f"Колонки: {world.columns.tolist()}")
    print(f"Система координат: {world.crs}")
    print(f"Типы геометрий: {world.geometry.geom_type.value_counts()}")
    return world


def task_1_2(world):
    print(world[['NAME', 'POP_EST', 'GDP_MD', 'geometry']].head())
    # Информация о России
    russia = world[world['NAME'] == 'Russia']
    print(f"\nРоссия: {russia.geometry.iloc[0].geom_type}")
    print(f"Граница России -- это {len(russia.geometry.iloc[0].geoms)} полигонов")
    # Простая визуализация
    fig, ax = plt.subplots(1, 1, figsize=(15, 10))
    world.plot(ax=ax, color='lightgreen', edgecolor='black')
    plt.title('Карта мира')
```

```
        plt.savefig('world_map.png', dpi=150)
        plt.show()


    def task_1_3():
        pipeline = WaterQualityPipeline("Практическая работа №1")
        # 1. Загружаем данные о странах

pipeline.load_world_data('data/ne_110m_admin_0_countries.shp')
        # 2. Создаем точки проб
        pipeline.create_sample_points(15, 'astrahan')
        # Оформляем результат
        pipeline.summary()
        return pipeline


    def task_2_1(pipeline: WaterQualityPipeline):
        print("\n" + "="*50)
        print("ДЕМОНСТРАЦИЯ РАБОТЫ С СК-42")
        print("="*50)
        sk42_points = pipeline.create_sk42_points(5)
        print("Точки в СК-42 созданы")
        print(sk42_points)
        points = pipeline.convert_to_wgs84()
        print(points)
        return sk42_points


    def task_2_2(pipeline: WaterQualityPipeline, sk42_points:
gpd.GeoDataFrame):
        pipeline.calculate_distances(sk42_points)


    def task_3_1(pipeline: WaterQualityPipeline):
        print("\n" + "="*50)
```

```
        print("ИССЛЕДОВАНИЕ ПЛОЩАДЕЙ")
        print("="*50)
        pipeline.compare_area_calculations()
        pipeline.plot_area_comparison()


    def task_4_1(pipeline: WaterQualityPipeline):
        region_name = 'astrahan'
        pipeline.create_sample_points(20, region_name)
        # 6. Карта с точками
        output_file = f'map_{region_name}.html'
        full_path = pipeline.output_dir + output_file
        pipeline.visualize_samples(output_file)  # передаем только
имя файла
        webbrowser.open('file://' + os.path.realpath(full_path))


    def task_4_2(pipeline: WaterQualityPipeline):
        saved_file                                                  =
pipeline.create_buffer_zones(buffer_size_km=4)
        full_path = pipeline.output_dir + saved_file
        webbrowser.open('file://' + os.path.realpath(full_path))


    def task_5_1(pipeline: WaterQualityPipeline):
        pipeline.save_results()
        pipeline.summary()



    def main():
        world = task_1_1()
        task_1_2(world)
        pipeline = task_1_3()
        sk42_points = task_2_1(pipeline)
```

```
        task_2_2(pipeline, sk42_points)
        task_3_1(pipeline)
        task_4_1(pipeline)
        task_4_2(pipeline)
        task_5_1(pipeline)


    if __name__ == '__main__':
        main()
```

Файл water_quality_pipeline.py:

```
    import geopandas as gpd
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    from shapely.geometry import Point
    import folium
    import os
    import utm
    import json
    import pickle


    class WaterQualityPipeline:
        def __init__(self, name="Мониторинг качества воды",
output_dir='class_data/'):
            self.name = name
            self.samples = None
            self.samples_sk42 = None
            self.world_data = None
            self.results = {}
            self.output_dir = output_dir
            import os
            os.makedirs(self.output_dir, exist_ok=True)
            print(f"Создан объект: {name}")
```

```
        def load_world_data(self, path='data/ne_110m_admin_0_countries.shp'):
            """Загрузка данных о странах мира"""
            self.world_data = gpd.read_file(path)
            print(f"Загружены данные о {len(self.world_data)} странах")
            return self.world_data


        def create_sample_points(self, n_points, region):
            """Создание тестовых точек проб воды"""
            if region == 'astrahan':
                lats = np.random.uniform(45.0, 46.0, n_points)
                lons = np.random.uniform(47.0, 49.0, n_points)
            else:
                raise ValueError("Неизвестный регион")

            data = pd.DataFrame({
                'id': range(1, n_points + 1),
                'lat': lats,
                'lon': lons,
                'name': [f'Точка_{i}' for i in range(1, n_points + 1)]
            })
            geometry = [Point(lon, lat) for lon, lat in zip(lons, lats)]
            self.samples = gpd.GeoDataFrame(
                data,
                geometry=geometry,
                crs='EPSG:4326'
            )
            points_file = os.path.join(self.output_dir, f'sample_points_{region}.geojson')
```

```
            self.samples.to_file(points_file, driver='GeoJSON')
            print(f"Создано {len(self.samples)} тестовых точек
проб")
            return self.samples


        def summary(self):
            """Краткая сводка о состоянии пайплайна"""

            print("\n" + "=" * 50)
            print(f"СОСТОЯНИЕ ПАЙПЛАЙНА: {self.name}")
            print("=" * 50)
            print(f"Пробы: {len(self.samples) if self.samples is
not None else 'не созданы'}")
            print(f"CRS проб: {self.samples.crs if self.samples
is not None else 'N/A'}")
            print(f"Данные мира: {'загружены' if self.world_data
is not None else 'нет'}")
            print(f"Результаты: {len(self.results)} сохраненных
результатов")
            print("=" * 50)


        #Методы для работы с координатами
        def create_sk42_points(self, n_points=5):
            """Создание тестовых точек в системе СК-42 (зона 7)
            Координаты в СК-42 (метры) для территории
Астраханской области"""

            x_coords = np.random.uniform(4970000, 5100000,
n_points)
            y_coords = np.random.uniform(8660000, 9350000,
n_points)
            data = pd.DataFrame({
                'id': range(1, n_points + 1),
```

```
                'x_sk42': x_coords,
                'y_sk42': y_coords,
                'name': [f'СК-42_точка_{i}' for i in range(1,
n_points + 1)]
            })
            geometry = [Point(y, x) for y, x in zip(y_coords,
x_coords)]
            self.samples_sk42 = gpd.GeoDataFrame(
                data,
                geometry=geometry,
                crs='EPSG:28408' # СК-42, зона 8
            )

            return self.samples_sk42


        def convert_to_wgs84(self):
            """ Преобразование проб в WGS84 """

            if self.samples_sk42 is None:
                raise ValueError("Нет данных для преобразования")

            original_crs = self.samples_sk42.crs
            self.samples = self.samples_sk42.to_crs('EPSG:4326')
            print(f"Координаты преобразованы из {original_crs} в
WGS84")
            return self.samples


        def calculate_distances(self, gdf):
            """
            Расчет расстояний между точками в разных проекциях
            """
            # Приводим точки к WGS84
            gdf = gdf.to_crs('EPSG:4326')
```

```
            # Определяем зону UTM по средней точке (38N или 39N
для Астраханской области)
            lon,        lat        =        gdf.geometry.x.mean(),
gdf.geometry.y.mean()
            zone_number = utm.from_latlon(lat, lon)[2]
            epsg_code  =  32600  +  zone_number    #  для  северного
полушария
            # Преобразуем в UTM (метры)
            gdf_utm = gdf.to_crs(f'EPSG:{epsg_code}')
            print(f"Зона UTM: {zone_number}N (EPSG:{epsg_code})")
            distances_utm = []
            distances_deg = []
            for i in range(len(gdf) - 1):
                p1_utm = gdf_utm.geometry.iloc[i]
                p2_utm = gdf_utm.geometry.iloc[i + 1]
                dist_utm = p1_utm.distance(p2_utm)
                distances_utm.append(dist_utm)
                p1_deg = gdf.geometry.iloc[i]
                p2_deg = gdf.geometry.iloc[i + 1]
                dist_deg = p1_deg.distance(p2_deg)
                distances_deg.append(dist_deg)

            distances_deg_km = [d * 111 for d in distances_deg]

            print("\nСравнение расстояний:")
            print("-" * 50)
            print("Пара | В UTM (м) | В градусах (км, грубо) |
Разница")
            print("-" * 50)
            for        i,        (d_utm,        d_deg_km)        in
enumerate(zip(distances_utm, distances_deg_km)):
                diff = abs(d_utm / 1000 - d_deg_km)
                print(f"{i  +  1}-{i  +  2}  |  {d_utm:.0f}  м  |
{d_deg_km:.2f} км | {diff:.2f} км")
```

```
        def compare_area_calculations(self):
            # Сравнение расчета площади России в разных проекциях
            if self.world_data is None:
                raise ValueError("Сначала загрузите данные о странах (load_world_data)")

            russia = self.world_data[self.world_data['NAME'] =='Russia'].copy()

            print("\nСРАВНЕНИЕ РАСЧеТА ПЛОЩАДИ РОССИИ")
            print("=" * 60)

            area_deg = russia.geometry.area.iloc[0]
            area_deg_km2 = area_deg * (111.32 * 111.32)

            print(f"В WGS84 (неправильно!): {area_deg_km2 / 1e6:.2f} млн км²")

            russia_moll = russia.to_crs('ESRI:54009')
            area_moll = russia_moll.geometry.area.iloc[0] / 1e12

            print(f"Равновеликая (Мольвейде): {area_moll:.2f} млн км²")

            russia_aea = russia.to_crs('ESRI:102013')
            area_aea = russia_aea.geometry.area.iloc[0] / 1e12

            print(f"Альберса (для России): {area_aea:.2f} млн км²")

            real_area = 17.13

            print("-" * 60)
```

```
            print(f"Реальная площадь России: {real_area} млн км²")
            print(f"Ошибка WGS84: {abs(area_deg_km2 / 1e6 - real_area) / real_area * 100:.1f}%")

            results = {
                'wgs84': area_deg_km2 / 1e6,
                'mollweide': area_moll,
                'aea': area_aea,
                'real': real_area
            }
            self.results['area_comparison'] = results

            return results

        def plot_area_comparison(self):
            # Визуализация сравнения площадей
            if 'area_comparison' not in self.results:
                raise ValueError("Сначала выполните compare_area_calculations()")

            areas = self.results['area_comparison']
            fig, ax = plt.subplots(figsize=(10, 6))
            methods = ['WGS84 (ошибочно)', 'Мольвейде', 'Альберса', 'Реальная']
            values = [areas['wgs84'], areas['mollweide'],
            areas['aea'], areas['real']]
            colors = ['red', 'green', 'green', 'blue']
            bars = ax.bar(methods, values, color=colors, alpha=0.7)
            ax.set_ylabel('Площадь (млн км²)')
            ax.set_title('Сравнение расчета площади России в разныхпроекциях')
            ax.axhline(y=areas['real'], color='blue', linestyle='--',
```

```
            label='Реальная площадь')
            ax.legend()

            for bar, val in zip(bars, values):
                height = bar.get_height()
                ax.text(bar.get_x()   +   bar.get_width()   /   2.,
height +
                        0.2,      f'{val:.2f}',      ha='center',
va='bottom')

            plt.tight_layout()
            plt.savefig('area_comparison.png', dpi=150)
            plt.show()
            print("График сохранен: area_comparison.png")


        def visualize_samples(self, filename='samples_map.html'):
            # Визуализация точек проб на интерактивной карте
            if self.samples is None:
                raise ValueError("Нет данных для визуализации")

            # Центр карты
            center_lat = self.samples.geometry.y.mean()
            center_lon = self.samples.geometry.x.mean()

            # Создаем карту
            m   =   folium.Map(location=[center_lat,   center_lon],
zoom_start=8)
            # Добавляем точки
            for idx, row in self.samples.iterrows():
                folium.Marker(
                    location=[row.geometry.y, row.geometry.x],
                    popup=f"ID:           {row['id']}<br>Название:
{row['name']}",
```

```
                    icon=folium.Icon(color='red',      icon='info-
sign')
                ).add_to(m)
            # Добавляем спутниковый слой
            folium.TileLayer(

tiles='https://server.arcgisonline.com/ArcGIS/rest/services/World_
Imagery/MapServer/tile/{z}/{y}/{x}',
                attr='Esri',
                name='Спутник'
            ).add_to(m)
            # Добавляем контроль слоев: переключатель
            folium.LayerControl().add_to(m)
            # Сохраняем
            m.save(self.output_dir+filename)
            print(f"Карта     сохранена:     {self.output_dir     +
filename}")

            return m


        def      create_buffer_zones(self,      buffer_size_km=4,
filename='buffer_zones_map.html'):
            # Создание буферных зон вокруг точек проб
            if self.samples is None:
                raise ValueError("Нет данных")

            # Определяем зону UTM по средней точке (38N или 39N
для Астраханской области)
            lon,     lat     =     self.samples.geometry.x.mean(),
self.samples.geometry.y.mean()
            epsg_code = 32600 + utm.from_latlon(lat, lon)[2]
            # Переходим в метрическую проекцию
            samples_utm                                          =
self.samples.to_crs(f'EPSG:{epsg_code}')
```

```
            # Создаем буферы (в метрах)
            buffer_size_m = buffer_size_km * 1000
            samples_utm['buffer'] =
samples_utm.geometry.buffer(buffer_size_m)
            # Создаем отдельный GeoDataFrame только с буферами

            buffers_gdf = gpd.GeoDataFrame(
                samples_utm[['id', 'name']], # берем нужные
атрибуты
                geometry=samples_utm['buffer'],
                crs=samples_utm.crs
            )

            # Преобразуем буферы в WGS84 для визуализации
            buffers_wgs84 = buffers_gdf.to_crs('EPSG:4326')
            # Центрируем карту по средним координатам исходных
точек
            center_lat = self.samples.geometry.y.mean()
            center_lon = self.samples.geometry.x.mean()
            m = folium.Map(location=[center_lat, center_lon],
zoom_start=8)

            # Добавляем буферы
            folium.GeoJson(
                buffers_wgs84,
                name='Буферные зоны',
                style_function=lambda x: {
                    'fillColor': 'blue',
                    'color': 'blue',
                    'weight': 1,
                    'fillOpacity': 0.1
                },
                tooltip=folium.GeoJsonTooltip(
                    fields=['id', 'name'],
                    aliases=['ID точки:', 'Название:'],
```

```
                    localize=True
                )
            ).add_to(m)

            # Добавляем точки
            for idx, row in self.samples.iterrows():
                folium.Marker(
                    location=[row.geometry.y, row.geometry.x],
                    popup=f"ID:           {row['id']}<br>Название:
{row['name']}",
                    icon=folium.Icon(color='red',      icon='info-
sign')
                ).add_to(m)

            # Добавляем спутниковый слой
            folium.TileLayer(

tiles='https://server.arcgisonline.com/ArcGIS/rest/services/World_
Imagery/MapServer/tile/{z}/{y}/{x}',
                attr='Esri',
                name='Спутник'
            ).add_to(m)
            # Добавляем контроль слоев
            folium.LayerControl().add_to(m)
            # Сохраняем
            m.save(self.output_dir + filename)
            print(f"Карта    с    буферными    зонами    сохранена:
{self.output_dir + filename}")

            return filename


        def save_results(self):
            results = {
                'name': self.name,
```

```
                'n_samples': len(self.samples) if self.samples is
not None else 0,
                'crs': str(self.samples.crs) if self.samples is
not None else None,
                'stats': {
                    'mean_lat':
float(self.samples.geometry.y.mean()) if self.samples is not None
else 0,
                    'mean_lon':
float(self.samples.geometry.x.mean()) if self.samples is not None
else 0,
                }
            }
            # Сохраняем JSON
            with open(self.output_dir + '1_results.json', 'w',
encoding='utf-8') as f:
                json.dump(results, f, ensure_ascii=False,
indent=2)

                if self.samples is not None:
                    self.samples.to_file(self.output_dir +
'work1_samples.geojson', driver='GeoJSON')

                print("Результаты сохранены:")
                print(" - work1_results.json")
                print(" - work1_samples.geojson")
                print(" - work1_samples_map.html")
                print(" - buffer_zones_map.html")
                print(" - area_comparison.png")
```